Bakingo PO Demand Forecasting — Final Handout

Author: generated for navajitkumardas@gmail.com **Date:** 2026-06-16 **Branch:** `claude/festive-faraday-ga6m86` **Repo path:** `/home/user/Automation`

---

1. Executive summary

We forecasted daily and weekly purchase-order demand for Bakingo's top-10 revenue SKUs and benchmarked **6 model families** plus **6 ensembles** plus **2 per-SKU model selectors** against a consistent 13-week (~90-day) holdout.

Final answer to ship to production today:

> **Weekly Croston (CrostonOptimized from `statsforecast`), refit per SKU, with a 13-week forecast horizon.**

That single recommendation produced the lowest mean sMAPE (39.2%) and the lowest mean aggregate quarterly error (35.5%) of any single model on the held-out fold. Every more sophisticated strategy we tried — global LightGBM, median ensembles, recency-weighted gated per-SKU selectors — either tied or lost on this dataset.

Where the real gains are next: data, not models. Add the store dimension (`unique_id = store × SKU`), festival regressors, and promo flags. Until then, model sophistication is hitting the noise floor.

---

2. Dataset

Property	Value
File	`bakingo_po_20240101_to_20260613.csv`
Rows	36,915 PO line items
Date range	2024-01-01 → 2026-06-12 (894 days, ~128 weeks)
Unique products	624
Unique stores	16
Total qty	904,926 units

Columns used: `createdAt` (date), `productName` (SKU identity), `qty` (volume target), `totalValue` (revenue target in INR). Other fields (store, customer, dispatch timestamp) were available but not used in this round.

Modeling targets

For each SKU we built **two daily aggregated series** — `qty` and `totalValue` — with missing days filled as zero, then in most analyses resampled to weekly buckets (Monday-start, W-MON).

Top 10 SKUs by historical revenue (₹)

#	SKU	Hist. ₹	Hist. qty
1	Choco Truffle Cake	4,309,145	32,334
2	Pineapple Cake	1,603,553	9,071
3	Fresh Fruit Cake	1,178,606	6,732
4	Ferrero Rocher Cake	1,011,030	3,047
5	Belgium Choco Mousse	1,001,520	3,005
6	Biscoff Jar Cake	786,736	7,110
7	Dream Cake	620,708	2,890
8	Tiramisu Cake	620,653	2,928
9	Butterscotch Cake	574,629	5,959
10	Choco Vanilla Cake	483,860	4,223

These 10 SKUs are what we report metrics on throughout. For the global LightGBM model the training panel was extended to the top 30 SKUs to give the model more cross-SKU signal.

---

3. Methodology

Holdout protocol

Single common holdout used for every benchmark: **the last 13 weeks of data** (2026-03-16 → 2026-06-08). All models trained on data strictly before that cutoff, then asked to predict the holdout. This makes all results directly comparable.

For the per-SKU model selector we extended this to multiple **rolling-origin folds** (4 CV folds + 1 held-out eval), so selection never sees the data it is graded on.

Error metrics

Metric	Meaning	Use for
MAE	mean absolute error (`y` -units)	day/week-level miss
RMSE	root mean sq. error	penalizes big misses
MAPE	mean abs % error (non-zero days)	relative miss
sMAPE	symmetric MAPE, bounded [0%, 200%]	sparse series — primary metric

Metric	Meaning	Use for
Bias	mean(pred − act)	over / under bias
SumPctErr	(sum_pred − sum_act) / sum_act × 100	aggregate (quarterly total) accuracy

We report **sMAPE** and **|SumPctErr|** as the two headline numbers.

Models benchmarked

Model	Library	What it is
Prophet	`prophet` (Meta)	trend + seasonality + holidays, daily or weekly
AutoARIMA	`statsforecast` (Nixtla)	auto-selected (S)ARIMA per SKU
AutoETS	`statsforecast`	auto-selected exponential smoothing per SKU
Croston	`statsforecast` (`CrostonOptimized`)	intermittent-demand specialist
LightGBM_global	`mlforecast` + LightGBM	global gradient-boosted model on lag/date features
6 Ensembles	—	mean / median over: all 5, no-Prophet, classical-only
Selector v1, v2	—	per-SKU model choice via CV

---

4. Headline 90-day forecast (Prophet, daily, default)

This was the first deliverable. Numbers come from `run_prophet_top_both.py`, fit on full history.

#	SKU	Forecast ₹ (90d)	Forecast qty (90d)
1	Choco Truffle Cake	574,069	4,125
2	Pineapple Cake	133,659	1,090
3	Fresh Fruit Cake	185,228	985
4	Ferrero Rocher Cake	241,048	755
5	Belgium Choco Mousse	92,101	360
6	Biscoff Jar Cake	55,764	640
7	Dream Cake	75,349	471
8	Tiramisu Cake	69,122	387
9	Butterscotch Cake	73,261	710
10	Choco Vanilla Cake	55,008	365

The backtest in §5–7 shows this Prophet forecast over-predicts most SKUs by 50-130% on the 90-day total — do not use it for planning. The recommended weekly Croston numbers below are the ones to use.

---

5. Backtest v1 — Prophet daily, 90-day holdout

`run_prophet_backtest.py` — same hold-out window, Prophet retrained without the last 90 days.

SKU / metric	Actual	Forecast	SumPctErr	sMAPE
Choco Truffle ₹	353,469	616,023	+74%	111%
Pineapple ₹	118,199	216,611	+83%	114%
Fresh Fruit ₹	117,904	170,722	+45%	117%
Ferrero Rocher ₹	175,206	203,346	+16%	134%
Belgium Choco Mousse ₹	59,414	94,125	+58%	127%
Biscoff Jar ₹	23,960	17,095	−29%	146%
Dream Cake ₹	67,150	61,562	−8%	141%
Tiramisu ₹	58,748	96,896	+65%	123%
Butterscotch ₹	38,161	89,263	+134%	124%
Choco Vanilla ₹	31,695	49,675	+57%	120%

Findings - Day-level sMAPE ~120% across the board — daily PO series are bursty (~50% zero-days) and Prophet cannot match the spike pattern. - Strong upward over-forecast bias on 8 of 10 SKUs. - Aggregate accuracy is only usable for a handful (Dream, Ferrero, Biscoff).

---

6. Backtest v2 — three Prophet configurations

`run_prophet_backtest_v2.py` — same holdout, three knobs tested: - **A** = daily, `cps=0.05` (default) - **B** = daily, `cps` tuned per SKU on a pre-holdout validation window - **C** = weekly aggregation (`W-MON`), `cps=0.05`

Config	avg sMAPE
A daily default	~125%
B daily tuned	~131%
C weekly	**~52%**

Single biggest insight of the whole project: weekly aggregation halves point-level error. The daily bursty noise gets averaged out at weekly granularity while trend and seasonality are preserved.

After this finding, **every subsequent benchmark uses weekly granularity.**

---

7. Five-model weekly benchmark — the headline table

Same 13-week holdout, weekly aggregation, top 10 SKUs scored on both metrics. From `run_statsforecast_compare.py` and `run_lightgbm_global.py`.

Model	Mean sMAPE	Median sMAPE	Mean |SumErr|	Median |SumErr|
Croston	**39.2%**	**32.8%**	**35.5%**	30.0%
AutoETS	40.8%	33.9%	39.2%	30.7%
AutoARIMA	41.6%	34.0%	35.9%	**27.7%**
LightGBM_global	41.6%	38.3%	39.6%	35.2%
Prophet	53.1%	47.7%	61.0%	50.6%

Findings - Three classical models (Croston, AutoETS, AutoARIMA) are statistically indistinguishable and **each beats Prophet by ~25% on point error and ~40% on aggregate error**. - LightGBM_global ties AutoARIMA — the global model isn't pulling its weight because the panel is small (30 SKUs × ~120 weeks ≈ 3.6k rows) and there are no exogenous features (store, promo, festival) for it to exploit. Its feature importance is dominated by `lag1`, `lag8`, `lag26` and rolling means — essentially what ARIMA already extracts. - **Drop Prophet from the stack.** It loses materially on every metric versus all four alternatives.

Per-SKU winners (10 SKUs × 2 metrics = 20 contests)

Model	Wins on individual contests
AutoARIMA	6
LightGBM	3
Croston	3
Prophet	3
AutoETS	2
(ensembles + ties)	3

Winners are heterogeneous — which motivated trying ensembles and a per-SKU selector.

---

8. Ensembles — do they help?

`run_ensemble_backtest.py`. Combined the 5 base models into six ensembles: mean and median over (all 5), (no Prophet), and (classical only — ARIMA + ETS + Croston).

Ensemble	Mean sMAPE	Median |SumErr|
Croston (single)	39.2%	30.0%
median_classical	**39.2%**	29.6%
median_no_prophet	39.4%	30.4%

Ensemble	Mean sMAPE	Median |SumErr|
mean_classical	39.7%	29.1%
mean_no_prophet	39.9%	31.8%
median_all (5)	40.0%	**28.9%**
mean_all	41.3%	35.4%

Findings - No ensemble overtakes Croston on mean sMAPE. When one model dominates, averaging it with weaker models pulls accuracy toward the mean. - The 5-model median wins on **median |SumErr| (28.9%)** — the safest bet for "don't blow the quarterly total". - Ensembles only win **3 of 20** individual contests — they're "average" by construction.

---

9. Per-SKU model selectors — the smart approach that didn't pay off

Two attempts at routing each SKU to its best model via CV.

v1 — `run_model_selector.py`

3 rolling-origin folds (2 CV + 1 eval), simple lowest-mean-sMAPE rule. **Result**: Selector got 43.6% mean sMAPE on the held-out fold (worse than always-Croston's 39.2%) but **best-in-class median |SumErr| at 25.2%**. Only 2 CV folds is too noisy a selection signal.

v2 — `run_model_selector_v2.py`

4 CV folds + 1 eval, recency-weighted (exponential weights 1, 2, 4, 8), safety gate: stay on default (Croston) unless candidate beats it by **≥ 5 pp sMAPE** in weighted CV.

Model	Mean sMAPE	Median |SumErr|
Croston	**39.2%**	30.0%
Selector_v2	41.6%	28.3%

The gate worked — kept 13/20 SKUs on Croston (where uncertain), preventing exactly the kind of disasters that hurt v1. But the 7 SKUs that switched: **3 wins, 1 tie, 3 large losses** (+22, +17, +12 pp on the held-out fold). Net: still ~2 pp behind always-Croston.

Switches that paid off	Switches that backfired
Pineapple ₹ → AutoARIMA (−2.6 pp)	Fresh Fruit ₹ → Prophet (+12.6 pp)
Fresh Fruit qty → Prophet (−5.2 pp)	Ferrero Rocher ₹ → Prophet (+2.6 pp)
Ferrero Rocher qty → Prophet (≈ tie)	Choco Vanilla ₹ → Prophet (+17.6 pp)
	Choco Vanilla qty → AutoARIMA (+22.7 pp)

Conclusion: With ~120 weeks of weekly data per SKU, even 4 recency- weighted CV folds and a conservative gate cannot reliably tell which model will win on the next 13 weeks. The variance of model rank across time is just too high.

10. Full leaderboard — every approach tried

#	Approach	Mean sMAPE	Median sMAPE	Median |SumErr|	Comment
1	**Croston (single)**	**39.2%**	32.8%	30.0%	Production choice
2	median_classical	39.2%	33.6%	29.6%	Statistical tie with Croston
3	median_no_prophet	39.4%	33.7%	30.4%	
4	mean_classical	39.7%	35.1%	**29.1%**	
5	mean_no_prophet	39.9%	33.5%	31.8%	
6	median_all (5 models)	40.0%	34.6%	**28.9%**	Best median |SumErr|
7	AutoETS	40.8%	33.9%	30.7%	
8	AutoARIMA	41.6%	34.0%	**27.7%**	Wins most individual SKUs
9	LightGBM_global	41.6%	38.3%	35.2%	Capped by data volume
10	Selector_v2 (gated)	41.6%	35.0%	28.3%	Gate prevents disasters
11	mean_all	41.3%	36.3%	35.4%	
12	Selector_v1	43.6%	35.5%	**25.2%**	Best median |SumErr|, but noisy
13	Prophet (weekly)	53.1%	47.7%	50.6%	Worst single model
14	Prophet (daily)	~125%	—	—	Don't use

11. Recommendations

Production v1 — ship now

```
Use: CrostonOptimized (statsforecast), weekly W-MON aggregation
Refit: per SKU, every Monday on all data
Horizon: 13 weeks
Inventory planning safety margin: +30% on quarterly totals
                                  (matches mean |SumErr| of 35.5%)
```

This is the **simplest, most accurate**, and **most robust** strategy we measured. ~30 lines of Python. Per-SKU sMAPE typically 30-50%, median quarterly error ~30%.

When to switch off Croston

- For SKUs with a clear yearly seasonality and a long history (Fresh Fruit, Choco Vanilla) — **AutoETS** is usually slightly better, but the difference is within noise.

- If you need a single point-level minimum across all SKUs, the **median of (ARIMA + ETS + Croston)** is essentially tied with Croston and is more robust against any one model failing — use it as v2 if operational complexity is acceptable.

Don't ship

- **Prophet** in any flavor on this dataset. 25%+ worse on every metric.
- **LightGBM_global as-is.** Need richer features and a wider panel before it earns its complexity.
- **Selector v1/v2** at current data scale. Re-evaluate after store dimension is added.

Special-case alert: Butterscotch Cake

Every model misses by +90% to +140% on the held-out window. This is **not a modeling failure — it's a real demand shift** at the training/ holdout boundary. Recommendations: - Investigate operationally: a SKU launch, recipe change, channel expansion, or promotion happened around mid-March 2026. - Either truncate training to the post-shift period, or add a manual changepoint indicator and refit.

---

12. Where the real gains are next (data, not models)

Model-side tuning is exhausted on this dataset. Ordered by expected ROI:

1. **Store dimension** — make the modeling key `store × SKU`. Panel goes from ~30 series to ~480 series. Global LightGBM finally has enough cross-series signal to dominate. Expected mean sMAPE: 25-35%.
2. **Festival calendar regressors** — weeks-to / weeks-from each major Indian festival (Diwali, Christmas, Valentine's, Rakhi, Holi, Mother's Day, Father's Day). Cake demand spikes hard on these and the current models don't see them.
3. **Promotion / pricing data** — usually the largest single demand driver in retail. If a promo table exists elsewhere, integrate it.
4. **Rolling-origin CV with more folds** — once the data is wider, re-run the per-SKU selector. It will start winning because individual models will pull apart from each other.

---

13. Repo layout

```
/home/user/Automation/
├── run_prophet.py                 # generic single-series Prophet
├── run_prophet_top.py             # top-N, one metric
├── run_prophet_top_both.py        # top-N, both metrics side-by-side
├── run_prophet_backtest.py        # baseline backtest
├── run_prophet_backtest_v2.py     # 3-config comparison
├── run_statsforecast_compare.py   # ARIMA/ETS/Croston vs Prophet
├── run_lightgbm_global.py         # global LightGBM panel model
├── run_ensemble_backtest.py       # 6 ensembles
├── run_model_selector.py          # per-SKU selector v1
```

```
├── run_model_selector_v2.py        # gated, recency-weighted v2
├── HANDOUT.md                      # this document
├── HANDOUT.pdf                     # printable handout
├── scripts/md_to_pdf.py            # regenerate the PDF
└── forecast_out/
    ├── forecast_ALL_PRODUCTS.{png,csv}
    ├── top/                # top-N (single metric)
    ├── top_both/           # top-N (both metrics)
    ├── backtest/           # baseline Prophet backtest
    ├── backtest_v2/        # 3-config comparison
    ├── statsforecast/      # ARIMA / ETS / Croston benchmark
    ├── lightgbm/           # global LightGBM panel
    ├── ensemble/           # 6 ensembles + per-SKU plots
    ├── selector/           # selector v1
    └── selector_v2/        # selector v2 (gated, weighted)
```

How to reproduce the final answer

```
pip install statsforecast pandas matplotlib
python3 run_statsforecast_compare.py --top 10 --holdout-days 90
# Croston metrics will be in forecast_out/statsforecast/_metrics_top10_compare.csv
```

How to refresh the PDF after editing this file

```
python3 scripts/md_to_pdf.py
```

---

14. Glossary

- **PO** — purchase order; each row of the input CSV is one product line on one PO.
- **`ds` / `y`** — column convention: timestamp and target.
- **Changepoint** — date where the underlying trend rate changes.
- **`changepoint_prior_scale` (cps)** — Prophet knob controlling trend flexibility. Higher = chase recent data; lower = smoother.
- **Holdout** — slice of recent data hidden from training and used for honest evaluation.
- **Rolling-origin CV** — repeated train-test splits where the train end rolls forward through time. Standard time-series cross-validation.
- **sMAPE** — symmetric MAPE. Bounded [0%, 200%]. Unlike MAPE it doesn't explode when actuals are zero — essential for sparse demand.
- **Intermittent demand** — series with many zero periods and irregular bursts. Croston's method is purpose-built for it.
- **Global model** — one model trained on many series at once (panel data), as opposed to one model per series.